

От **Ogre3D** к вебу: Пошаговый портирование **3D**-игры с **Emscripten**, от настройки до сетевого взаимодействия

Подготовка среды и настройка системы сборки

Процесс конвертации десктопной игры, написанной на C++, в исполняемый файл для веб-браузера является многоэтапным и требует тщательной подготовки рабочего окружения и точной настройки систем сборки. Этот начальный этап закладывает фундамент для всего последующего процесса и определяет, насколько гладко пройдет трансляция исходного кода в формат, совместимый с JavaScript и WebAssembly. Ключевой технологией, обеспечивающей этот переход, является Emscripten — набор инструментов, который позволяет компилировать код, написанный на C/C++, в высокооптимизированный JavaScript или нативный WebAssembly [14](#). Основное преимущество этого подхода заключается в возможности переноса существующего кода без его полного рефакторинга, что позволяет сохранить логику игры и использовать мощь уже существующего игрового движка, такого как Ogre3D [14](#).

Первым шагом в подготовке является установка официального набора инструментов Emscripten, известного как **emsdk**. Этот инструмент управляет установкой, обновлением и удалением различных версий компилятора Emscripten, необходимых утилит и JavaScript-библиотек, которые он использует [10](#). Процесс установки обычно сводится к клонированию репозитория **emsdk** из официального GitHub-проекта и выполнению скрипта установки. После завершения установки необходимо активировать созданное окружение. Это делается с помощью команды `source ./emsdk_env.sh`, которая должна выполняться в каждой новой терминальной сессии перед началом работы с Emscripten [10](#). Эта команда модифицирует переменные окружения, в частности **PATH**, чтобы исполняемый файл компилятора **emcc** стал доступен глобально. **emcc** — это не настоящий компилятор, а Python-скрипт, который координирует весь процесс сборки, включая вызов LLVM-компилятора для преобразования C++ в промежуточное представление (IR), а затем трансляцию этого IR в

JavaScript или WebAssembly с помощью специальных утилит Emscripten, таких как `asm2wasm` и `jsify`.

После успешной установки и активации Emscripten следующим критически важным шагом является настройка системы сборки проекта. Большинство современных C++ проектов используют CMake в качестве системы управления сборкой. Для того чтобы CMake могла работать с Emscripten, необходимо создать специальный файл конфигурации, называемый "toolchain file". Этот файл содержит все необходимые параметры, указывающие CMake на то, что целевой платформой является Emscripten. Имя файла может быть любым, но общепринятым соглашением является `cmake-toolchain.cmake`. В этом файле определяются ключевые переменные, такие как `CMAKE_SYSTEM_NAME`, которое должно быть установлено в Emscripten, и `CMAKE_C_COMPILER`, `CMAKE_CXX_COMPILER`, которые должны указывать на исполняемый файл `emcc`. Кроме того, в этом файле можно задавать специфические для Emscripten флаги компиляции и линковки, например, `EMSCRIPTEN_ABI_MANGLE` для контроля над именами функций в экспортируемом коде. Альтернативный, более простой способ — установить переменную окружения `EMSCRIPTEN_TOOLCHAIN_FILE` и указать путь к этому файлу. При вызове CMake для генерации проектных файлов необходимо передать флаг `-DCMAKE_TOOLCHAIN_FILE=путь/к/вашему/cmake-toolchain.cmake`. Когда CMake видит, что `CMAKE_SYSTEM_NAME` равен Emscripten, она автоматически настраивает все пути и флаги для правильной работы с Emscripten. Важно также явно указать тип выходного файла: для игры это будет исполняемый файл, который Emscripten при компиляции преобразует в `.js` и `.wasm` файлы. Для библиотек следует использовать соответствующие типы, например, `.so` (разделяемая библиотека).

В некоторых случаях может потребоваться возможность проверять внутри кода, запущенного в браузере, на какой именно платформе он работает. Emscripten предоставляет для этого механизм, основанный на макросах и глобальных объектах. После запуска скомпилированного приложения в JavaScript-контексте становится доступным глобальный объект `Module`. Разработчик может определить объект `Module` до загрузки скрипта игры, передав ему различные параметры, например, информацию о ресурсах или поведении приложения. Внутри C++ кода можно проверить наличие определенных макросов, которые Emscripten определяет во время компиляции, чтобы отличить веб-сборку от нативной. Например, можно использовать условную компиляцию `#ifdef EMSCRIPTEN` для включения или исключения платформенно-зависимого кода. Это особенно полезно для функционала, который имеет разные реализации на

разных платформах, например, для загрузки файлов или получения путей к директориям. Unity, хотя и использует свой собственный плагин для WebGL, демонстрирует важность наличия механизма идентификации платформы, что является общим принципом для любой кросс-платформенной разработки [11](#).

Для успешной сборки сложного проекта, такого как игра на Ogre3D, могут потребоваться дополнительные зависимости. Например, Ogre3D может зависеть от таких библиотек, как FreeImage, OpenEXR, Zlib и других. Все эти зависимости также должны быть собраны с помощью Emscripten. Если используется `emsdk`, можно легко установить и скомпилировать многие популярные библиотеки с помощью менеджера пакетов `emscripten`. Процесс обычно включает в себя получение исходного кода библиотеки, создание соответствующего `CMakeLists.txt` файла (если его нет) и его последующую сборку с помощью `emcc`. Необходимо убедиться, что пути к заголовочным файлам и скомпилированным библиотекам этих зависимостей корректно указаны в системе сборки основного проекта. Это гарантирует, что компилятор и линкер смогут найти все необходимые для игры компоненты. Некоторые зависимости могут иметь свои особенности сборки для Emscripten, поэтому важно обращать внимание на документацию и примеры, предоставляемые самими разработчиками библиотек. Сравнение версий пакетов между различными дистрибутивами, как это делается в `apt`, помогает понять, какие версии библиотек доступны и совместимы [7](#).

Таким образом, подготовка среды и настройка системы сборки — это не просто механический процесс, а стратегическое мероприятие, которое определяет успех всего проекта. Установка `emsdk`, активация окружения, создание файла конфигурации CMake и корректное управление зависимостями являются обязательными шагами. Тщательная настройка этой основы позволит в дальнейшем сосредоточиться на решении более сложных задач, связанных с адаптацией самого движка Ogre3D и реализацией специфического игрового функционала, не беспокоясь о том, что базовый компилятор или система сборки будут препятствовать прогрессу.

Интеграция и адаптация движка **Ogre3D** для **WebGL**

Интеграция движка Ogre3D в проект, предназначенный для запуска в веб-браузере через Emscripten, является наиболее сложным и критически важным

этапом всего процесса портирования. Основная трудность заключается в том, что Ogre3D, будучи кросс-платформенным движком, изначально спроектирован для использования нативных операционных системных API для выполнения своих задач. Эти API включают в себя систему управления окнами (Win32, X11, Cocoa), системные вызовы для рендеринга (Direct3D, OpenGL) и другие низкоуровневые взаимодействия с оборудованием, которые недоступны в изолированной среде веб-браузера. Простая попытка скомпилировать исходный код Ogre3D с помощью компилятора Emscripten (emcc) без дополнительных усилий неминуемо приведет к ошибкам, поскольку компилятор столкнется с невиданными ранее функциями и заголовочными файлами, специфичными для конкретной операционной системы.

Поэтому ключевой задачей становится обеспечение совместимости Ogre3D с веб-средой. Существует два основных подхода к решению этой проблемы: использование готовой, предварительно собранной версии Ogre3D, адаптированной для Emscripten, и самостоятельная сборка и модификация исходного кода движка. Первый подход является наиболее предпочтительным и экономит значительное количество времени и сил. Сообщество Emscripten и разработчики сторонних библиотек часто создают и поддерживают сборки популярных фреймворков для веб-платформы. Перед тем как приступить к сложному процессу самостоятельной сборки, необходимо провести тщательный поиск готовых, работающих сборок Ogre3D. Такие сборки обычно уже содержат все необходимые адаптеры, исправления и настройки, чтобы движатель мог корректно инициализироваться и работать в среде Emscripten. Они могут быть доступны в виде пакетов через менеджеры зависимостей, таких как vcpkg или conan, или же распространяться через сообщества и форумы, посвященные Emscripten и веб-геймингу. Если такая сборка найдена, процесс значительно упрощается: достаточно скопировать каталоги с заголовочными файлами (`include`) и скомпилированные библиотеки (`lib`) в структуру проекта и правильно настроить систему сборки (CMake), чтобы она ссылалась на эти пути.

Если же готовой сборки не существует, разработчику придется прибегнуть ко второму, гораздо более трудоемкому подходу — самостоятельной сборке и модификации Ogre3D. Этот путь требует глубокого понимания как самого движка Ogre3D, так и экосистемы Emscripten. Процесс начинается с получения исходного кода Ogre3D и всех его зависимостей (FreeImage, OpenEXR, etc.). Затем каждая из этих зависимостей должна быть скомпилирована для Emscripten, что само по себе может быть нетривиальной задачей и требует создания соответствующих скриптов сборки. После того как все зависимости

собраны, можно приступать к сборке самого Ogre3D. Для этого нужно модифицировать его систему сборки на основе CMake, чтобы она использовала Emscripten в качестве инструментария. Однако даже после успешной компиляции библиотек движка, скорее всего, потребуется внести изменения непосредственно в исходный код Ogre3D. Эти изменения направлены на замену нативных системных вызовов на абстракции, предоставляемые Emscripten. Например, необходимо переопределить классы, отвечающие за управление окном и циклом приложения, чтобы они не использовали Win32 или X11, а вместо этого интегрировались с функциями Emscripten, такими как `emscripten_set_main_loop`, которая служит главным игровым циклом в браузере. Также потребуется адаптировать часть классов, связанных с рендерингом, например, `Ogre::GLRenderSystem`, чтобы они корректно работали с WebGL через JavaScript-интерфейсы, предоставляемые Emscripten.

Центральным аспектом интеграции является преобразование графических вызовов. Emscripten эмулирует OpenGL ES API, позволяя C++ коду, использующему стандартные функции OpenGL, работать в браузере. Когда код компилируется через Emscripten, вызовы OpenGL ES транслируются в соответствующие вызовы JavaScript-API WebGL [11](#). Это означает, что если игра была написана с использованием профиля OpenGL ES 2.0, она сможет работать в большинстве современных браузеров, которые предоставляют контекст WebGL 1.0, являющийся веб-портированием OpenGL ES 2.0. Однако современные браузеры поддерживают и более новые стандарты, такие как WebGL 2.0, который основан на OpenGL ES 3.0 [15](#). Использование WebGL 2.0 открывает доступ к продвинутым возможностям, таким как текстуры с другим форматом данных, многократное сэмплирование и compute-шейдеры. Emscripten способен предоставлять контекст WebGL 2.0, однако, как показывает опыт разработчиков Unity, иногда требуется вручную изменить внутренние файлы Emscripten, например `library_browser.js`, чтобы принудительно запрашивать контекст `webgl2` вместо `webgl` по умолчанию [10](#). Это важный практический нюанс, который необходимо учитывать при сборке. Хотя compute-шейдеры, доступные в WebGL 2.0, пока не полностью поддерживаются Emscripten, использование WebGL 2.0 остается предпочтительным для игр, требующих более высокого уровня графики [15](#).

Ниже представлена таблица, сравнивающая два основных подхода к интеграции Ogre3D.

Критерий	Использование готовой сборки	Самостоятельная сборка
Сложность	Низкая	Очень высокая
Временные затраты	Минимальные	Значительные (часы/дни)
Требуемые знания	Базовые знания CMake и управления зависимостями	Глубокое понимание CMake, Emscripten, системного программирования и архитектуры Ogre3D
Стабильность	Высокая (если сборка актуальна и проверена)	Зависит от качества модификаций; высокий риск ошибок
Поддержка	Зависит от сообщества/автора сборки	Полностью зависит от разработчика проекта
Ключевые действия	Поиск, скачивание, настройка путей в CMake	Модификация исходников, сборка зависимостей, настройка CMake

Вне зависимости от выбранного подхода, после успешной интеграции и компиляции Ogre3D для Emscripten, разработчик получает библиотеку, которая может инициализировать графический контекст (WebGL), создавать сцены, модели, материалы и отрисовывать их в `<canvas>` элементе на HTML-странице. Это создает прочную основу для дальнейшей реализации игровой логики и интеграции других необходимых функций, таких как управление вводом, звук и сеть. Успешная адаптация графической подсистемы движка является залогом того, что 3D-графика игры будет корректно отображаться в веб-браузере, обеспечивая ожидаемый пользовательский опыт.

Реализация управления пользовательским вводом

После успешной интеграции графической подсистемы Ogre3D и обеспечения корректного отображения 3D-сцены, следующим критически важным шагом является реализация управления пользовательским вводом. Для веб-игры, как и для любой другой, необходима возможность реагировать на действия пользователя с клавиатуры и мыши. В отличие от десктопных приложений, где эти события обрабатываются нативными системными вызовами, в среде браузера все взаимодействие происходит через JavaScript-интерфейсы API (Application Programming Interface). Emscripten предоставляет мощный и удобный механизм для моста между этими двумя мирами, позволяя C++ коду напрямую подписываться на события ввода и получать доступ к их данным.

Основной принцип работы Emscripten с событиями ввода заключается в регистрации функций обратного вызова (callback functions) на уровне JavaScript. Emscripten предоставляет набор функций C-стиля, которые позволяют разработчику указать, какая функция на C++ стороне должна быть вызвана при возникновении определенного события, такого как нажатие клавиши (keydown), отпускание клавиши (keyup), движение мыши (mousemove) или клик мыши (mousedown). Эти функции обратного вызова становятся доступными после выполнения скомпилированного кода и позволяя получить доступ к состоянию клавиш и движения мыши прямо внутри C++.

Для реализации обработки ввода не требуется никаких сторонних библиотек; вся необходимая функциональность встроена в Emscripten. Процесс настройки обычно включает в себя несколько шагов. Во-первых, в коде игры необходимо определить функции C-стиля, которые будут служить обработчиками событий. Эти функции должны объявляться с использованием спецификатора extern "C", чтобы избежать проблем с декорированием имен (name mangling), и принимать соответствующие параметры, описывающие событие. Например, обработчик для события клавиши может принимать параметры, указывающие на код клавиши и является ли это повторным нажатием. Во-вторых, после инициализации приложения, но до начала основного игрового цикла, необходимо вызвать функции Emscripten для регистрации этих обработчиков. Например, для клавиатуры используется функция `emscripten_set_keydown_callback`, которой в качестве аргументов передается HTML-элемент, на который вешается коллбек (например, `EMSCRIPTEN_EVENT_TARGET_DOCUMENT` для отслеживания клавиш по всей странице), флаг, указывающий, нужно ли блокировать стандартное поведение браузера, и указатель на функцию-обработчик. Аналогичные функции существуют для других типов событий: `emscripten_set_mousemove_callback` для движения мыши, `emscripten_set_mousedown_callback` и `emscripten_set_mouseup_callback` для кликов.

После регистрации обработчиков, данные о событиях будут автоматически передаваться из JavaScript-слоя в C++ код. Внутри функции-обработчика можно получить доступ к информации о событии, такой как код клавиши (scancode, keycode), символ, который она представляет, состояние модификаторов (Shift, Ctrl, Alt) и координаты курсора мыши. Эта информация может быть немедленно использована для обновления игровой логики: перемещения камеры, управления персонажем, открытия меню и т.д. Emscripten также предоставляет удобные функции для получения текущего состояния

клавиш (например, `emscripten_get_key_down`), что позволяет проверять, зажата ли определенная клавиша в любой момент времени, а не только при событии нажатия.

Преимущество такого подхода заключается в его инкапсуляции и простоте. Разработчик может продолжать писать логику управления игры на C++, используя привычные для него конструкции, не заботясь о деталях работы с DOM-событиями в JavaScript. Emscripten берет на себя всю сложную работу по маппингу браузерных событий в вызовы C++-функций. Это позволяет сохранить чистоту кода и разделить ответственность: JavaScript-часть Emscripten-обертки занимается взаимодействием с браузером, а основной игровой код на C++ — обработкой логики игры.

Для мыши, помимо кнопок, важна также информация о ее положении относительно холста и величине движения. Emscripten предоставляет функции для получения этих данных. Например, можно узнать абсолютные координаты курсора с помощью `EM_ASM`, а также получить информацию о перемещении мыши с помощью обработчиков `mousemove`. Это позволяет реализовать такие игровые механики, как вращение камеры при зажатой правой кнопке мыши или отслеживание курсора.

Важно отметить, что при работе с вводом в браузере необходимо учитывать контекст. Например, когда пользователь взаимодействует с элементами управления (кнопками, полями ввода), которые могут находиться на той же странице, что и игра, важно правильно настроить фокус и обработку событий, чтобы игра не "забирала" все ввод. Emscripten позволяет настроить целевой элемент для обработки событий, что дает гибкость в управлении этим поведением.

В конечном счете, реализация управления вводом с помощью встроенных API Emscripten является прямым, надежным и эффективным решением. Она позволяет быстро и без дополнительных зависимостей добавить в игру полноценную поддержку клавиатуры и мыши, сохранив при этом основной код на C++. Этот шаг является неотъемлемой частью процесса портирования, превращая статичное 3D-приложение в интерактивную игру, способную реагировать на действия пользователя.

Интеграция звуковой подсистемы

Интеграция звуковой подсистемы является еще одним ключевым аспектом, который превращает 3D-приложение в полноценную игру. В отличие от графики, где Emscripten предоставляет эмуляцию OpenGL ES, для аудио в веб-среде отсутствует единого стандарта, аналогичного DirectX Audio или Core Audio. Вместо этого браузеры предоставляют собственные, мощные API, такие как Web Audio API, который является стандартом де-факто для воспроизведения звука в браузере. Поэтому задача портирования звукового функционала сводится к тому, чтобы найти или создать слой абстракции на C/C++, который сможет использовать этот веб-стандарт. Наиболее эффективным и простым решением для этой задачи является использование легковесной, кросс-платформенной библиотеки **miniaudio**.

Miniaudio — это однозагрузочная (single-file) библиотека на языке C, специально разработанная для простого и эффективного воспроизведения звука [16](#) [17](#). Ее ключевыми преимуществами, делающими ее идеальным кандидатом для портирования через Emscripten, являются: 1. Легковесность и простота интеграции: Библиотека состоит всего из двух файлов — заголовочного (`miniaudio.h`) и исходного (`miniaudio.c`). Это позволяет легко добавить ее в любой проект, не создавая сложных зависимостей или систем сборки. 2. Встроенная поддержка **Emscripten**: Miniaudio изначально спроектирована с учетом кросс-платформенности и имеет встроенный backend для Emscripten. Этот backend использует Web Audio API для воспроизведения звука в браузере, что делает его идеально совместимым с экосистемой Emscripten. 3. Простой и интуитивно понятный **API**: Интерфейс программирования приложений miniaudio очень прост и ориентирован на решение базовых задач: инициализация устройства вывода, загрузка аудиоданных (из файлов или памяти) и их воспроизведение. Это позволяет быстро внедрить звуковой функционал, не тратя время на изучение сложных концепций. 4. Открытый исходный код и активная поддержка: Проект miniaudio находится под свободной лицензией и активно развивается. Существует множество документации, примеров и руководств, включая анализ поддержки многопоточности на Emscripten, что может быть полезно для более сложных сценариев [18](#).

Процесс интеграции miniaudio в проект, скомпилированный с помощью Emscripten, выглядит следующим образом. Во-первых, необходимо добавить файлы `miniaudio.h` и `miniaudio.c` в структуру проекта. Во-вторых, в системе сборки (CMake) нужно добавить путь к этому каталогу в директиву

`include_directories` и добавить `miniaudio.c` в список исходных файлов для компиляции вместе с основным кодом игры. Поскольку `miniaudio` поддерживает Emscripten, компилятор `emcc` автоматически выберет правильный backend во время сборки. В-третьих, необходимо модифицировать код игры. Логика инициализации звука должна быть инкапсулирована в блок условной компиляции, который будет компилироваться только для веб-платформы. Например, можно создать функцию `init_audio()` в C++, которая при вызове для Emscripten-сборки инициализирует экземпляр `ma_device` из `miniaudio`, указывая в параметрах, что устройство вывода должно быть Emscripten-совместимым. `Miniaudio` сам позаботится о создании необходимых объектов Web Audio API (`AudioContext`, `ScriptProcessorNode`) и их связывании с буфером звуковых данных.

После инициализации устройства вывода, оставшаяся часть кода игры может оставаться практически неизменной. Вместо вызовов нативных функций для проигрывания звука, теперь будут вызываться функции API `miniaudio`, такие как `ma_decoder_init_file` для загрузки звука из файла и `ma_device_start` для запуска воспроизведения. `Miniaudio` предоставляет удобные классы для декодирования различных форматов звуковых файлов (WAV, MP3, FLAC и др.) и управления ими. Весь звуковой поток будет передаваться из C++ кода в браузер через `miniaudio` и далее в Web Audio API, где он будет смешан с другими источниками звука на странице и отправлен на аудиовыход пользователя.

Важно понимать, что хотя `miniaudio` и является отличным решением, оно несет в себе некоторые ограничения, присущие самой технологии Web Audio API. Например, управление объемом и панорамированием звука осуществляется через объекты Web Audio API, и для сложных трехмерных звуковых эффектов может потребоваться более глубокое взаимодействие с этим API, возможно, через прямой вызов JavaScript-функций из C++ с помощью Emscripten's `EM_JS` макросов. Однако для большинства игровых задач, таких как фоновая музыка и звуковые эффекты, API `miniaudio` является достаточным и удобным.

Альтернативные подходы, такие как попытка использовать другие аудио-библиотеки, которые могут не иметь официальной поддержки Emscripten, могут привести к большим проблемам. Разработчику придется самостоятельно писать адаптер, который бы переводил вызовы этой библиотеки в вызовы Web Audio API, что является нетривиальной задачей. Использование `miniaudio` позволяет избежать этого, так как эта работа уже сделана профессионально и тщательно протестирована сообществом.

Таким образом, для интеграции звуковой подсистемы в портируемую игру на Ogre3D настоятельно рекомендуется использовать библиотеку `miniaudio`. Этот выбор основан на ее легковесности, простоте интеграции, встроенной поддержке Emscripten и простоте использования, что в совокупности значительно упрощает процесс портирования и позволяет быстро добавить в игру необходимый звуковой функционал.

Настройка сетевого взаимодействия через WebSocket

Реализация сетевого взаимодействия является финальным и одним из самых важных шагов для превращения одиночной игры в онлайн-продукт. Для веб-платформы, ограниченной современными браузерами, стандартом для создания постоянных, низколатентных каналов связи между клиентом и сервером является протокол WebSocket. В отличие от традиционного HTTP-запроса, который является однонаправленным и требует повторной установки соединения для каждого взаимодействия, WebSocket устанавливает долгоживущее, полудуплексное (двустороннее) соединение. Это делает его идеальным выбором для таких приложений, как онлайн-игры, где важна скорость и минимизация задержек [3](#) [5](#). Emscripten предоставляет богатый набор инструментов для работы с сетью, позволяя C/C++ коду использовать как эмулированные BSD Sockets, так и полноценный WebSocket API.

Emscripten предлагает два основных уровня абстракции для сетевых взаимодействий. Первый, более низкоуровневый, — это эмуляция BSD Socket API. Emscripten может транслировать стандартные вызовы сокетов (`socket()`, `connect()`, `send()`, `recv()`) в реализацию на основе XMLHttpRequest (XHR) или Fetch API [3](#). Это позволяет существующему C/C++ коду, который был написан с использованием сокетовой модели для TCP, работать с веб-серверами без необходимости переписывания сетевой логики. Однако для многих игровых сценариев, особенно для многопользовательских игр, где требуется постоянное общение с сервером, этот подход может быть менее эффективным из-за накладных расходов, связанных с эмуляцией TCP поверх HTTP. Более того, браузеры не предоставляют прямого доступа к сокетам TCP, что делает WebSocket наиболее близким к нему протоколом, доступным в вебе [5](#).

Второй, и более предпочтительный для игр, уровень абстракции — это полноценный WebSocket API. Emscripten предоставляет C/C++ заголовочный файл `emscripten/websocket.h`, который определяет функции и структуры для работы с WebSocket ¹. Этот API позволяет C/C++ коду устанавливать WebSocket-соединение, отправлять текстовые и бинарные данные, а также обрабатывать события, такие как открытие соединения, получение сообщения или закрытие соединения. Все это происходит на уровне C/C++, при этом Emscripten берет на себя детали взаимодействия с JavaScript-WebSocket API.

Процесс интеграции WebSocket в игру выглядит следующим образом. В первую очередь, в коде игры необходимо включить заголовочный файл `emscripten/websocket.h`. Затем, для установления соединения, вызывается функция `emscripten_websocket_new`, которой передается URL сервера. Эта функция возвращает дескриптор WebSocket-соединения, который используется в последующих вызовах. Для отправки данных используется функция `emscripten_websocket_send`, которая принимает дескриптор, указатель на буфер данных и его размер. Для приема данных необходимо реализовать обработчик событий. Emscripten позволяет зарегистрировать функцию обратного вызова, которая будет вызываться при поступлении новых данных. Эта функция получает дескриптор соединения, а также указатель на буфер с данными и его размер. Полученные данные можно передавать дальше в игровую логику для обработки.

Одним из ключевых преимуществ использования WebSocket API Emscripten является его надежность и простота в использовании. Разработчики сообщества уже создали примеры реализации расширенных функций, таких как механизм автоподключения. Автоматическое переподключение — это критически важная функция для онлайн-игр, которая позволяет игроку снова подключиться к серверу после временной потери сети, не перезагружая всю игру. Реализация такого механизма в Emscripten включает в себя регистрацию обработчиков событий `close` и `error` для WebSocket-соединения. Внутри этих обработчиков можно запустить логику повторного подключения с экспоненциальной задержкой, чтобы не перегружать сервер ⁴. Такие примеры, доступные в открытых источниках, значительно упрощают создание отказоустойчивого сетевого кода.

Важно понимать, что при работе с сетью в вебе необходимо учитывать политику безопасности CORS (Cross-Origin Resource Sharing). Сервер, с которым пытается соединиться игра, должен отправлять правильные заголовки,

разрешающие доступ с домена, на котором размещена игра. Это настраивается на стороне серверного приложения и не зависит от кода на стороне клиента.

В заключение, для реализации сетевого взаимодействия в портированной игре на Ogre3D настоятельно рекомендуется использовать WebSocket API, предоставляемый Emscripten. Этот подход обеспечивает надежное, низколатентное соединение, которое является основой для большинства онлайн-игр. Он позволяет писать сетевой код на C++, сохраняя при этом чистоту и структуру существующего проекта, и при этом предоставляет доступ к мощным механизмам, таким как автоподключение, для повышения надежности и качества пользовательского опыта.

Финальная сборка, развертывание и тестирование

Финальный этап портирования игры на Ogre3D в веб-платформу завершается процессом сборки, развертывания и всестороннего тестирования. Этот этап объединяет все предыдущие усилия и приводит к созданию конечного продукта, который может быть запущен в любом современном веб-браузере. Процесс сборки с помощью Emscripten уникален по сравнению с нативной компиляцией и требует понимания структуры выходных файлов и их роли в работе приложения.

Процесс сборки запускается через основной компилятор Emscripten, `emcc`. Вместо того чтобы создавать один исполняемый файл, `emcc` генерирует набор файлов, которые вместе составляют веб-приложение. Ключевыми из них являются: 1. **.js файл (JavaScript-обертка)**: Это главный файл приложения. Он содержит не сам игровой код, а JavaScript-обертку, которая управляет жизненным циклом приложения. Эта обертка отвечает за инициализацию виртуальной файловой системы, загрузку ресурсов, создание `<canvas>` элемента для рендеринга, а также за связку C++ кода с браузерными API (WebGL, Web Audio, WebSocket). Она также содержит JavaScript-код, который запускает и управляет выполнением скомпилированного в WebAssembly кода. 2. **.wasm файл (WebAssembly)**: Это основной исполняемый код вашего приложения. Именно он содержит всю логику игры, математические вычисления, физику, искусственный интеллект и вызовы к движку Ogre3D. WebAssembly обеспечивает производительность, близкую к нативной, благодаря аппаратному ускорению, доступному в современных браузерах [14](#). 3. **.data**

файл (Данные ресурсов): Если в проекте используются внешние файлы, такие как текстуры, модели, шейдеры или конфигурационные файлы, Emscripten может упаковать их в этот файл. Обертка `.js` будет отвечать за их распаковку и монтирование в виртуальную файловую систему приложения во время запуска. 4. Дополнительные файлы: В зависимости от флагов сборки могут генерироваться и другие файлы, например, `.mem` для предвыделения памяти или карты отладки.

Для запуска собранного приложения необходим простой HTML-файл. Этот файл должен содержать тег `<script>`, который подключает основной JS-файл, сгенерированный Emscripten. Внутри этого HTML-файла должен быть также `<canvas>` элемент, который будет использоваться для рендеринга 3D-графики. Emscripten-обертка автоматически найдет этот холст по его идентификатору и начнет в него рисовать. Важно убедиться, что CSS-стили для `<canvas>` и всего документа настроены так, чтобы холст занимал всю доступную область окна браузера, если это требуется для игры.

После того как приложение собрано и настроен HTML-файл, наступает очередь тестирования. Это критически важный этап, на котором выявляются и исправляются проблемы, возникающие только в веб-среде. Тестирование должно проводиться в нескольких современных браузерах (Google Chrome, Mozilla Firefox, Microsoft Edge, Apple Safari), так как их реализации Web APIs могут иметь незначительные различия. Основные аспекты, на которые следует обратить внимание:

- **Производительность:** Необходимо проанализировать производительность игры. Emscripten-сборки могут быть больше по размеру, чем нативные, что влияет на время загрузки. Размер `.wasm` файла может быть значительным, и для его оптимизации могут потребоваться техники, такие как сжатие Gzip/Brotli, а также разгрузка ресурсов. Профилирование производительности с помощью инструментов разработчика в браузерах поможет выявить узкие места в коде, которые могут замедлять игру.
- **Совместимость:** Проверить корректность работы всех функций: отображение графики, управление вводом, воспроизведение звука, сетевое взаимодействие. Особое внимание стоит уделить совместимости графики. Если игра использует возможности WebGL 2.0, необходимо убедиться, что она корректно запрашивает и использует этот контекст в браузере, который его поддерживает [15](#). В случае проблем с контекстом WebGL, может потребоваться ручное вмешательство в файлы Emscripten, как было описано ранее [10](#).

- **Размер экрана и масштабирование:** Проверить, как игра ведет себя при изменении размера окна браузера. Убедиться, что разрешение холста корректно обновляется и что изображение не искажается.
- **Сетевые условия:** Если игра многопользовательская, необходимо протестировать ее в условиях плохого интернет-соединения, чтобы убедиться в работоспособности механизма автоподключения и общей отказоустойчивости сети ⁴.

В заключение, финальная сборка, развертывание и тестирование представляют собой комплексный процесс, требующий внимания к деталям. Корректная настройка HTML-файла, понимание назначения каждого из сгенерированных Emscripten файлов и тщательное тестирование в различных условиях являются залогом успешного выпуска портированной игры. Успешное прохождение этого этапа означает, что все предыдущие шаги по настройке среды, интеграции движка и реализации функционала были выполнены правильно, и конечный продукт готов к использованию миллионами пользователей в их веб-браузерах.

Справка

1. Emscripten中的WebSocket服务器: libwebsockets移植案例 https://blog.csdn.net/gitblog_00110/article/details/152703002
2. The WebSocket API (WebSockets) - Web APIs - MDN Web Docs https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API
3. 突破浏览器限制: Emscripten实现高效Web网络通信的完整方案 https://blog.csdn.net/gitblog_00174/article/details/152678854
4. Emscripten中的WebSocket客户端: 自动重连实现 https://blog.csdn.net/gitblog_01006/article/details/152705191
5. Emscripten - 网络支持 <https://blog.csdn.net/csdddnd/article/details/153726136>
6. Blend4Web. User Manual: Release v17.06.2 | PDF | Web Gl <https://www.scribd.com/document/467797314/b4w-manual-en-pdf>
7. Compare Packages Between Distributions <https://distrowatch.com/dwres.php?resource=compare-packages&firstlist=freebsd&secondlist=debian&firstversions=0&secondversions=6&showall=yes>

8. 平台和工具 - 架构师研究会 <https://architect.pub/book/export/html/36>
9. NLP Assignment <https://www.kaggle.com/mihirprajapati01/irs-assignment>
10. When requesting WebGL2, any browser gives me a ... <https://stackoverflow.com/questions/64298166/when-requesting-webgl2-any-browser-gives-me-a-webgl1-context-emsripten>
11. WebGL native plug-ins for Emscripten <https://docs.unity3d.com/2022.3/Documentation/Manual/webgl-native-plugins-with-emsripten.html>
12. How to use the .js and .wasm artifacts emitted by ... <https://stackoverflow.com/questions/67823329/how-to-use-the-js-and-wasm-artifacts-emitted-by-emsripten-and-embed>
13. OpenCV4.0 Change Logs 翻译 <https://blog.csdn.net/maweifei/article/details/83070167>
14. 终极指南：如何在Emscripten中优化WebGL扩展性能与兼容性 https://blog.csdn.net/gitblog_00748/article/details/152706007
15. What are the latest versions of OpenGL ES supported by ... <https://stackoverflow.com/questions/70483149/what-are-the-latest-versions-of-opengl-es-supported-by-emsripten-and-how-do-i>
16. miniaudio终极指南：10分钟上手跨平台音频开发 https://blog.csdn.net/gitblog_00114/article/details/153759144
17. 深入解析miniaudio：轻量级跨平台音频库的核心特性与应用 https://blog.csdn.net/gitblog_00630/article/details/148523710
18. miniaudio项目中的Emscripten多线程支持解析 <https://blog.gitcode.com/ad64201f8eb993a4dbb5b58524e9adad.html>